

AI Agents for Data Visualization and Kaggle Competitions

An end-to-end LLM agent (my-agent) benchmarked against two frozen yardsticks on 20 Kaggle competitions — with the apparatus needed to make the comparison honest

Author: Wei-Hao Huang · **School:** National Chengchi University, Department of Economics
Advisor: Dr. Tso-Jung Yen, Institute of Statistical Science, Academia Sinica · Summer Internship 2026

1 · Motivation

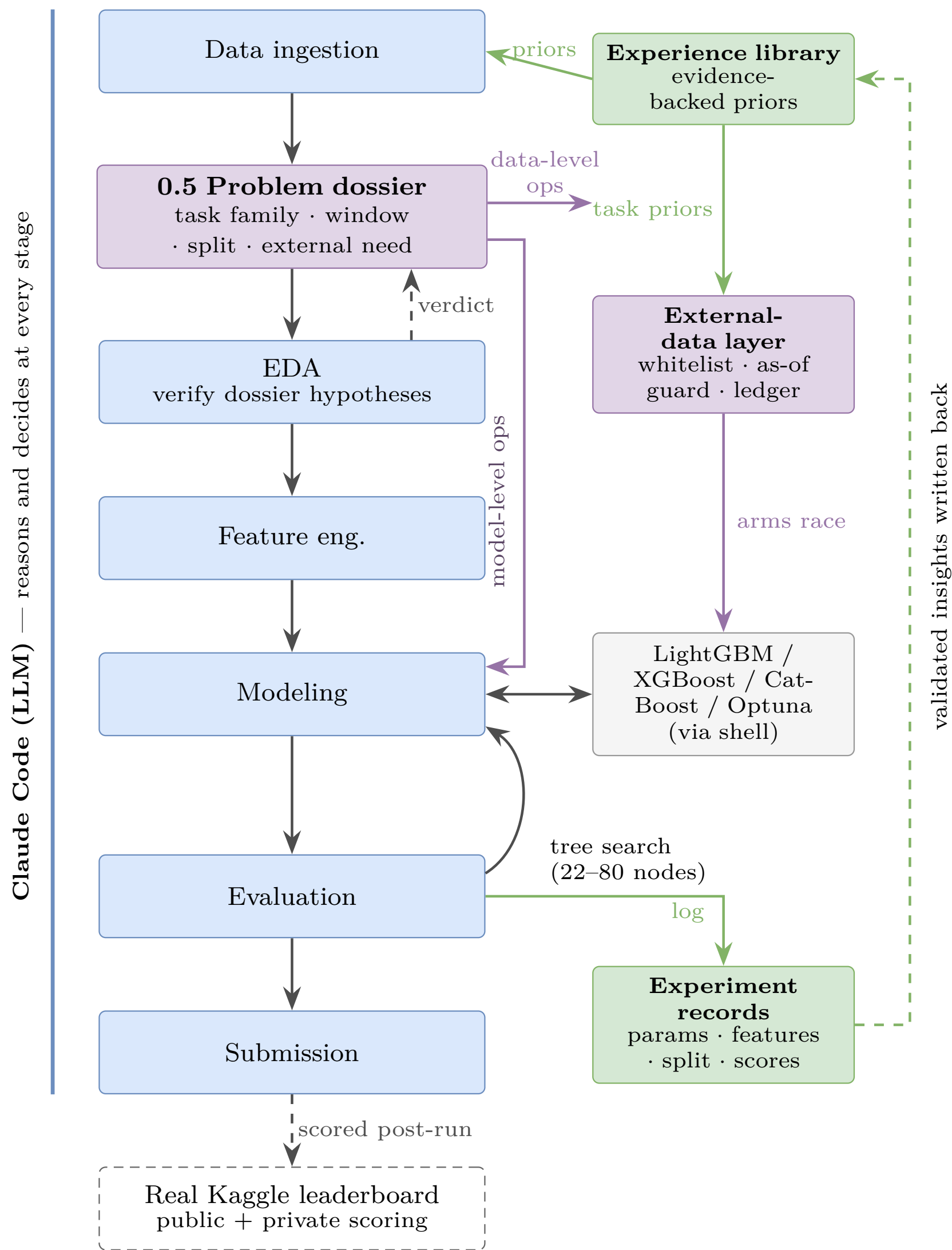
Why these tasks are hard

- **Kaggle competitions** demand the full expert loop — validation design, feature engineering, model selection, ensembling — where the difference between top entries sits at the **4th–5th decimal** of the metric.
- With gaps that small, **local cross-validation is an unreliable compass**: a pipeline's own validation score routinely disagrees with the real leaderboard, so no local number can be trusted to pick a winner.
- **Data visualization** (EDA, diagnostics, reporting) is essential but repetitive: every dataset needs distribution, drift, and correlation views before modeling can be trusted.

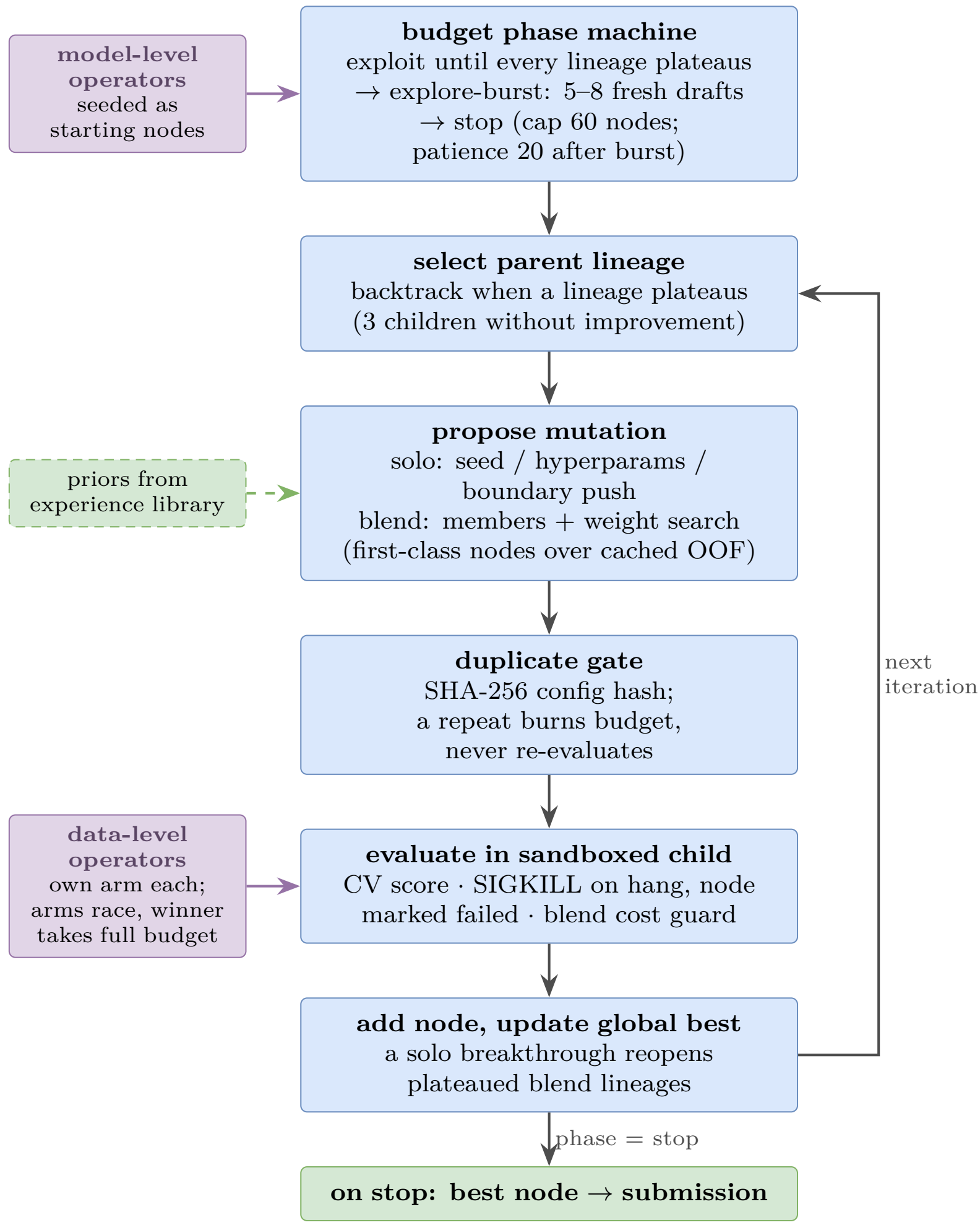
Why an AI agent

- Tireless, systematic search: 22–80 candidate configurations per competition, every experiment logged and reproducible.
- Runs the same disciplined protocol on 20 competitions serially — a human cannot hold this consistency.
- Generates per-dataset EDA figures and per-competition reports automatically, so every claim is grounded in an artifact.

2 · Workflow



The pipeline, read top to bottom: the LLM reasons at every stage; the **Stage-0.5 dossier** (injection layer) classifies the task and injects external data *upstream*, before any modeling; submissions are scored post-run on the real leaderboard. Colour = role: blue LLM pipeline · purple injection layer · green evidence loop · gray Auto-ML tools. Solid arrows = execution flow; dashed = feedback, advisory, or post-run steps.



Tree-search loop: ensembles are first-class nodes with OOF caching and weight search; exploit until plateau, then one forced exploration burst; the loop terminates at the best-node submission. Purple = the injection layer's operator entry points; green = experience-library priors.

3 · Results — Kaggle benchmark

Benchmark lanes scored on the real leaderboard (late submission); the mid-complexity three-way is offline-graded on the held-out private split (late submission structurally unavailable there).

Benchmark standing (scored 2026-08-18, 16 of 20 lanes)

All 20 competitions run under the frozen Stage-0.5 architecture in credential-free sandboxes with a per-lane transcript audit. The 16 completed lanes are scored on the real leaderboard, each duel adjudicated by a paired test built from Kaggle's own public/private split: **my-agent leads the standing — 15 wins / 5 losses (75% excl. ties)** vs. AIDE 42% and NVIDIA 38% — takes the **best private score on 10 of 16**, and beats its own development-lane score on 11 of 16. One lane running (s6e2), three queued (**TBD**).

Best private score across the 16 scored competitions

my-agent		10 / 16
NVIDIA		5 / 16
AIDE		1 / 16

Head-to-head on the same 16 real leaderboards: my-agent posts the best private score twice as often as both yardsticks combined. On s3e19, the architecture's canonical failure case, the injection layer's external-data join turns last place (SMAPE 50.455 without the layer) into a numerical first among the three agents (48.243) — and all **6 of 6** controlled external-data arms beat their matched baselines.

Mid-complexity three-way (offline-graded, private split)

Competition	Metric	my-agent	NVIDIA	AIDE
us-patent (NLP)	Pearson $r \uparrow$	0.86658	0.86160	0.65821
ventilator (time series)	MAE $\downarrow$	0.16525	0.40093	0.34591
SIIM-ISIC (imaging)	AUC $\uparrow$	0.93529	0.89822	0.88579

4 · Advantages & Difficulties

Expected → experienced

- **Expected:** the agent would quickly beat most humans. **Experienced:** development runs landed above average, with wins concentrated where no mature public solutions exist; the frozen-architecture re-run now leads the paired standing on the 16 scored lanes.
- **Expected:** one run and one score decide a winner. **Experienced:** score noise and run-to-run variance leave many duels undecidable; a paired error model became mandatory.
- **Expected:** the agent follows the rules by default. **Experienced:** a transcript audit caught one lane reading its own competition's recorded findings — and the clean re-run's local validation scored *worse*, consistent with the contamination advantage being real.
- Fairness is expensive: isolated data roots, credential-free sandboxes, and per-lane transcript audits cost real engineering time — but without them the numbers are not comparable.

5 · Discussion — advice

- **State the error model with every claim.** A ranking without an error scale is noise: many of our duels are undecidable, and the winner flips when the evaluation set changes.
- **Never trust local CV for between-agent claims.** Score on the real leaderboard; CV describes a pipeline's internal view only.
- **Freeze your yardsticks.** Improving a baseline mid-study silently turns the benchmark into an optimisation of the baseline.
- **Isolate mechanically, then audit the transcript.** Sandboxes stop leaks you anticipated; reading what the agent actually did catches the ones you didn't — in both directions (we hit one real contamination and one auditor false positive).
- **Put knowledge injection upstream.** Injected at the blend stage it was worth $\sim 10^{-5}$; moved into problem identification it lifted the canonical failure case from last to first.

Where knowledge enters

- Downstream (blend stage): measured gain $\sim 10^{-5}$ — abandoned.
- Upstream (problem dossier): fires on exactly the **2/18** competitions that need external data; all **6 of 6** controlled external-data arms beat their matched baselines.
- But no local signal can select the *form* of injection — the one surviving selector died on its first pre-registered prediction.

Companion documents: Report v6 (Overleaf, main document) · Engineering Log · Case Studies · 22 per-competition ML-spec reports.